



# INSTITUTO POLITÉCNICO NACIONAL

## Escuela Superior de Cómputo



### **Carrera:**

Ingeniería en Sistemas Computacionales

### **Unidad de Aprendizaje:**

Análisis y Diseño de Algoritmos

### **Actividad:**

Práctica 1.2 Medición del tiempo en Javascript

### **Nombre de los Alumnos:**

Álvarez Tahuilán Luis Gustavo

Noyola Gómez Emilio Damian

### **Nombre del Maestro:**

Ricardo Felipe Díaz Santiago

**Turno:** Matutino

**Grupo:** 3CM1

**Fecha de Entrega:** 17 de septiembre de 2026

## Práctica 1.2 Medición del tiempo en Javascript

### Introducción

El análisis de algoritmos permite estudiar el comportamiento de un programa en función del tamaño de los datos de entrada y observar cómo varía su eficiencia conforme aumenta la cantidad de elementos que debe procesar. En esta práctica se analiza nuevamente el algoritmo de Ordenamiento por Selección *Selection Sort*, un método que recorre de manera repetida la parte no ordenada de un arreglo para localizar el elemento menor y colocarlo en la posición que le corresponde.

En esta ocasión, el algoritmo se implementó en lenguaje JavaScript y se evaluó utilizando arreglos de diferentes tamaños, desde 100 hasta 1,000,000 de elementos. Para realizar las pruebas se consideró un arreglo en orden inverso como el peor de los casos, con el propósito de observar el comportamiento del algoritmo al incrementar progresivamente el tamaño de la entrada. El tiempo de ejecución se midió mediante la función *performance.now()*, la cual permite obtener mediciones de tiempo de alta resolución expresadas en milisegundos.

A partir de los tiempos obtenidos para cada tamaño de arreglo, se organizan y grafican los resultados con el fin de analizar experimentalmente la relación existente entre el número de elementos y el tiempo requerido.

*El objetivo final es comprobar experimentalmente el comportamiento cuadrático del Ordenamiento por Selección y analizar cómo el incremento en el tamaño de los datos de entrada afecta directamente su tiempo de ejecución.*

### Desarrollo

#### I. Explicación del funcionamiento

```
1  function CArray(numElements) {
2      this.dataStore = [];
3      this.pos = 0;
4      this.numElements = numElements;
5      this.insert = insert;
6      this.toString = toString;
7      this.clear = clear;
8      this.setData = setData;
9      this.setWorstCase = setWorstCase;
10     this.swap = swap;
11     this.selectionSort = selectionSort;
12
13     for (var i = 0; i < numElements; ++i) {
14         this.dataStore[i] = i;
15     }
16 }
```

`CArray()` se encarga de inicializar un nuevo objeto que representa y maneja el arreglo. Dentro de esta función, se definen propiedades clave mediante la palabra clave `this`, como `dataStore` para almacenar físicamente los datos (inicializado como un arreglo vacío), `pos` para controlar la posición de inserción actual y `numElements` para guardar el tamaño configurado de la entrada.

Asocia funciones externas como `insert`, `selectionSort` o `swap` como métodos del objeto actual utilizando el operador `this`. De esta manera, una vez que se crea una instancia del objeto con `new CArray(...)`, es posible llamar a estos métodos directamente sobre ella. Finalmente, la función incluye un ciclo que llena el arreglo inicial secuencialmente con números desde 0 hasta `numElements - 1` como punto de partida para las pruebas de ordenamiento.

```
18 function setData() {
19     for (var i = 0; i < this.numElements; ++i) {
20         this.dataStore[i] = Math.floor(
21             Math.random() * (this.numElements + 1)
22         );
23     }
24 }
25
26 function setWorstCase() {
27     for (var i = 0; i < this.numElements; ++i) {
28         this.dataStore[i] = this.numElements - i;
29     }
30 }
31
32 function clear() {
33     for (var i = 0; i < this.dataStore.length; ++i) {
34         this.dataStore[i] = 0;
35     }
36 }
37
38 function insert(element) {
39     this.dataStore[this.pos++] = element;
40 }
```

La función `setData` se utiliza para llenar el `dataStore` con números pseudoaleatorios utilizando los objetos integrados `Math.random` y `Math.floor`, escalando los valores para que estén dentro del rango del tamaño de los elementos configurados. Por otro lado, `setWorstCase` genera la entrada de "peor caso" requerida para la práctica, llenando el arreglo en orden inverso (del número más grande al más pequeño) para probar el algoritmo bajo las condiciones más desafiantes.

Adicionalmente, se muestran las funciones `clear`, que recorre el arreglo para poner todos sus elementos a cero sin eliminar la estructura ni cambiar su tamaño, e `insert`. Esta última función añade un elemento

individual a la posición actual indicada por la variable `this.pos`, utilizando la sintaxis de post-incremento para insertar el valor y actualizar automáticamente el puntero de la posición para la siguiente operación de inserción.

```
42 function toString() {
43     var retstr = "";
44
45     for (var i = 0; i < this.dataStore.length; ++i) {
46         retstr += this.dataStore[i] + " ";
47
48         if (i > 0 && i % 10 == 0) {
49             retstr += "\n";
50         }
51     }
52
53     return retstr;
54 }
55
56 function swap(arr, index1, index2) {
57     var temp = arr[index1];
58     arr[index1] = arr[index2];
59     arr[index2] = temp;
60 }
```

La función `toString` se encarga de concatenar todo el contenido del arreglo en una sola cadena de texto formateada para su impresión en consola. Utiliza un ciclo para recorrer el `dataStore`, añadiendo espacios entre los números e insertando caracteres de salto de línea `\n` aproximadamente cada 10 elementos mediante el operador módulo `%`, lo que mejora la legibilidad de arreglos grandes cuando se muestran como texto.

A continuación, se muestra la función `swap`, la cual implementa el mecanismo clásico para intercambiar dos valores de posición dentro del arreglo de manera eficiente. Esta función recibe el arreglo en sí y los dos índices a intercambiar. Utiliza una variable temporal llamada `temp` para guardar el primer valor antes de sobrescribir con el segundo, y luego coloca el valor guardado en `temp` en la segunda posición, operación fundamental para el funcionamiento correcto de los algoritmos de ordenamiento.

```

62 function selectionSort() {
63     var min;
64
65     for (var outer = 0; outer < this.dataStore.length - 1; ++outer) {
66         min = outer;
67
68         for (var inner = outer + 1; inner < this.dataStore.length; ++inner) {
69             if (this.dataStore[inner] < this.dataStore[min]) {
70                 min = inner;
71             }
72         }
73
74         if (min !== outer) {
75             this.swap(this.dataStore, outer, min);
76         }
77     }
78 }

```

La función emplea dos ciclos anidados: el ciclo externo `outer` avanza posición por posición desde el principio del arreglo, indicando el lugar donde se colocará el siguiente valor mínimo. Al inicio de cada iteración externa, se asume que la posición actual contiene el mínimo, guardando el índice de esta posición en la variable `min`.

El ciclo interno `inner` comienza desde la posición siguiente a `outer` (`outer + 1`) y recorre el resto de la parte no ordenada del arreglo para buscar el elemento más pequeño restante y actualizar el índice guardado en la variable `min` si encuentra uno menor. Una vez terminada la búsqueda interna, si el índice del mínimo encontrado es diferente a la posición actual de `outer`, se llama al método `this.swap` asociado al objeto para realizar el intercambio, garantizando que el elemento más pequeño de la sección no ordenada se mueva a su lugar definitivo.

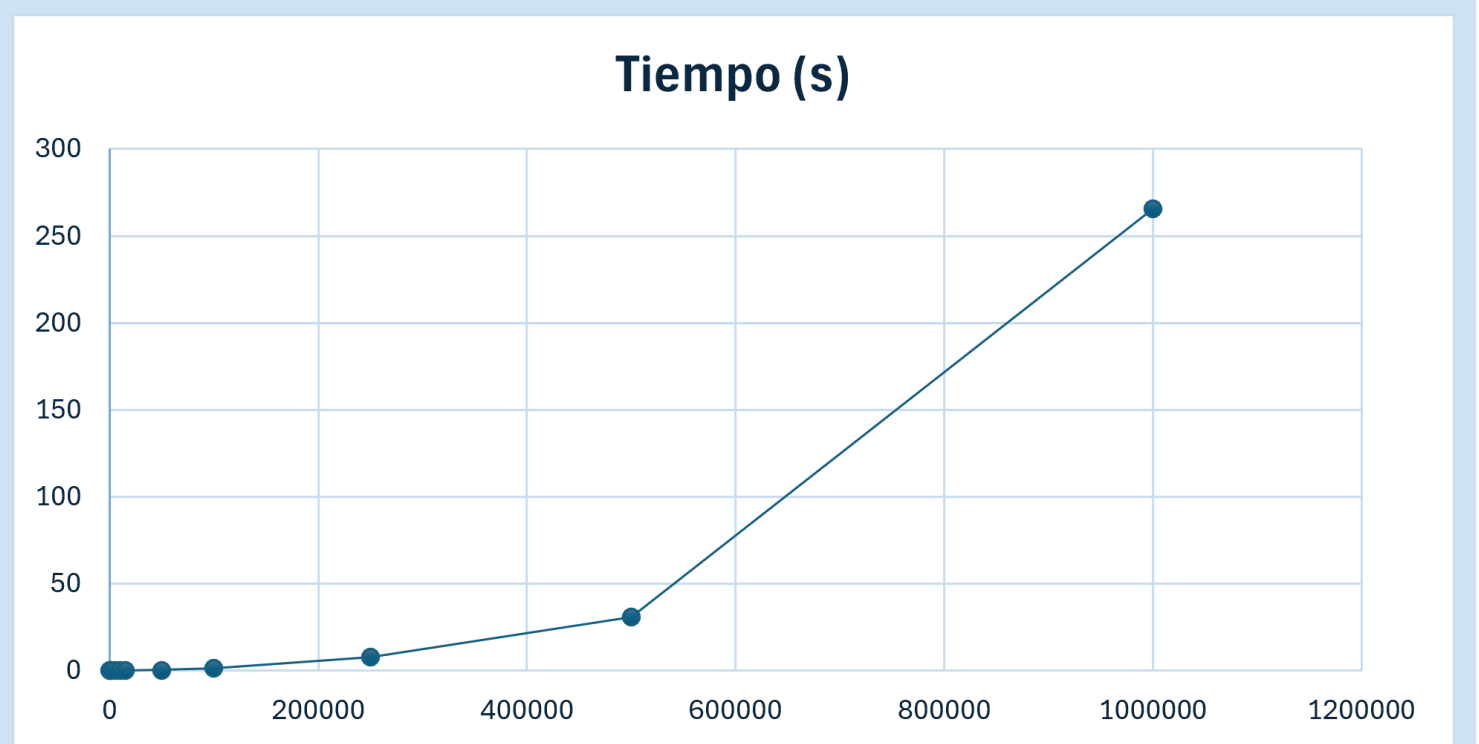
## II. Resultado de las Pruebas

| Elementos                  | Consola | Fuentes | Red | Rendimiento | Memoria | Aplicación | Seguridad | Lighthouse | Grabadora | 1 | 1 | 2 | 1 oculto  |
|----------------------------|---------|---------|-----|-------------|---------|------------|-----------|------------|-----------|---|---|---|-----------|
|                            |         |         |     |             |         |            |           |            |           |   |   |   |           |
| Elementos,Tiempo_ms        |         |         |     |             |         |            |           |            |           |   |   |   | VM105:94  |
| 100,0.3999999985098839     |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 500,0.8999999985098839     |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 1500,2.199999999254942     |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 5000,20.799999997019768    |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 10000,25.400000002235174   |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 15000,41.69999999925494    |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 50000,309.6000000014901    |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 100000,1227.199999999255   |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 250000,7692.699999999255   |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 500000,30982.300000000745  |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |
| 1000000,265473.80000000075 |         |         |     |             |         |            |           |            |           |   |   |   | VM105:111 |

Se listan los pares de datos correspondientes a cada tamaño de entrada probado (especificados en la variable `elementos` del código original, desde 100 hasta 1,000,000 elementos) y el tiempo exacto en milisegundos que tarda el algoritmo `Selection Sort` en ordenarlos bajo la prueba de peor caso.

Se puede observar claramente cómo el tiempo de ejecución aumenta de forma drástica a medida que crece el tamaño del arreglo de entrada. Por ejemplo, mientras que 100 elementos se ordenan en fracciones de milisegundo, la entrada de 1,000,000 elementos requiere aproximadamente 265,473 milisegundos (más de 4 minutos). Esta relación exponencial refleja la complejidad temporal cuadrática ( $O(n^2)$ ) intrínseca del algoritmo de ordenamiento por selección para entradas inversamente ordenadas.

### III. Gráficas de Pruebas



El eje horizontal  $X$  representa el número de elementos (el tamaño de la entrada), y el eje vertical ( $Y$ ) muestra el `Tiempo (s)` (Tiempo en segundos, transformado de los milisegundos mostrados en la consola) que tardó la ejecución. La línea une los puntos de datos, comenzando casi plana cerca del origen para entradas pequeñas y curvándose pronunciadamente hacia arriba para entradas más grandes.

Esta curva ascendente de crecimiento acelerado es la visualización característica de la complejidad temporal cuadrática ( $O(n^2)$ ) del algoritmo `Selection Sort`. Demuestra cómo el tiempo necesario para resolver el problema no aumenta linealmente con el tamaño de la entrada, sino en proporción al cuadrado del número de

elementos, reflejando cómo el rendimiento del algoritmo decae rápidamente al aumentar la escala del problema.

## Conclusiones

---

### Álvarez Tahuilán Luis Gustavo

Al realizar las pruebas fue posible notar la diferencia entre entender la complejidad de un algoritmo de forma teórica y verla reflejada en una ejecución real. Los resultados mostraron que el tamaño de la entrada influye de manera importante en el desempeño de *Selection Sort*, especialmente en los arreglos más grandes. La práctica permitió comprender mejor por qué el análisis de complejidad es útil para comparar algoritmos y anticipar su comportamiento antes de trabajar con grandes volúmenes de información.

### Noyola Gómez Emilio Damian

La práctica permitió observar de manera clara que *Selection Sort* deja de ser conveniente cuando el tamaño de los datos aumenta demasiado. Aunque su funcionamiento es sencillo de comprender e implementar, el crecimiento de sus operaciones hace que los tiempos aumentan rápidamente en entradas grandes. Esto ayudó a entender que no basta con que un algoritmo funcione correctamente, sino que también es importante considerar su eficiencia antes de utilizarlo en problemas reales.

## Bibliografía

---

- Brassard, G., & Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall.
- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1998). *Estructuras de datos y algoritmos*. Pearson Educación/Addison Wesley.